Synopsys ICC2 commands for Physical Design

1. FLOORPLAN STAGE

In Synopsys ICC2, the floorplan stage is the very first physical design step after synthesis. It involves defining the chip's dimensions, the core area for standard cells, placing large blocks (macros/IPs) and I/O pins, and establishing the initial power distribution network. This stage critically impacts subsequent steps like placement and routing.

Here's a breakdown of the typical ICC2 commands used in the floorplan stage, from start to end:

1. Initial Setup and Design Loading:

- **read_db <design_name>.db** / **read_verilog <design_name>.v**: Load the design database (after synthesis) or the Verilog netlist. The .db format is generally preferred as it contains more pre-optimized information.
- **read_sdc <constraints_file>.sdc**: Load the timing constraints (SDC) file. While not directly driving physical dimensions, understanding the timing budget helps in making informed floorplan decisions.
- **read_upf <power_intent_file>.upf (Optional, for multi-voltage designs)**: Load the Unified Power Format (UPF) file. This is crucial for multi-voltage designs as it defines power domains, voltage levels, and isolation requirements, which influence macro placement and power grid design.
- **set_attribute [find / -all -is_black_box true] black_box false**: If any modules were marked as black boxes during synthesis, this command can be used to treat them as hierarchical blocks rather than abstract black boxes for floorplanning.
- **current_design <top_level_design_name>**: Set the current design context to the top-level module.

2. Defining the Die and Core Area (Chip Outline):

- **initialize_floorplan**: This is the primary command to start defining the floorplan. It offers various options:
  - **-control_type {core | aspect_ratio | chip}**:
    - **core**: Defines the core area and lets the tool calculate the die size based on core-to-die spacing.
    - **aspect_ratio**: Defines the aspect ratio of the core area, and the tool calculates dimensions based on utilization or side lengths.
    - **chip**: Defines the total die size, and the tool calculates the core area based on chip-to-core spacing.
  - **-core_utilization <value>**: Specifies the target utilization for standard cells within the core area (e.g., 0.7 for 70%). This is crucial for initial area estimation.
  - **-shape {rectangular | square}**: Defines the shape of the core area.
  - **-orientation {N | S | E | W}**: Sets the initial orientation of the core or chip (relevant for I/O placement).
  - **-side_length {<x_len> <y_len>}**: Directly specifies the width and height of the core or chip.

    - **-core_offset {<left> <bottom> <right> <top>}**: Defines the spacing between the die boundary and the core boundary.
    - **-die_offset {<left> <bottom> <right> <top>}**: Defines the spacing between the core boundary and the die boundary (if control_type is core).
    - **-site_row_direction {horizontal | vertical}**: Specifies the preferred direction of standard cell rows.
    - **-start_origin {<x> <y>}**: Sets the origin of the floorplan.
- **set_attribute [find / -all -is_macro true] macro_cell true**: Ensures large blocks are correctly identified as macros.

3. Macro (IP Block) Placement:

- **place_macro**: Places individual macros or a list of macros.
    - **-origin {<x> <y>}**: Places the macro at a specific absolute coordinate.
    - **-relative_to <ref_cell>**: Places the macro relative to another cell.
    - **-halo {<left> <bottom> <right> <top>}**: Creates a keep-out zone (halo/margin) around the macro for routing channels.
    - **-side {top | bottom | left | right}**: Places the macro along a specific side of the core.
    - **-snap_to_grid {true | false}**: Aligns macro placement to the manufacturing grid.
    - **-orientation {R0 | R90 | MX | MY | MXR90 | MYR90}**: Sets the orientation of the macro.
    - **[get_cells <macro_name>]**: Specifies the macro to be placed.
- **place_pins_on_grid**: Places pins of macros on a grid, if not already done.
- **create_keepout_margin -type {hard | soft} -outer {<left> <bottom> <right> <top>} [get_cells <macro_name>]**: Explicitly defines keep-out margins around macros to prevent standard cells from being placed too close.
- **create_placement_blockage -name <blockage_name> -boundary { {llx lly} {urx ury} } -type {hard | soft | partial}**: Creates regions within the core area where standard cells should not be placed (hard), or where cell density should be reduced (soft/partial). Useful for anticipated routing congestion or sensitive areas.
- **create_bound -name <bound_name> -type {soft | hard | exclusive} -boundary { {llx lly} {urx ury} } [get_cells <cells_to_bound>]**: Defines regions where specific standard cells are encouraged (soft) or forced (hard/exclusive) to be placed. Used for critical modules or to group related logic.

4. I/O Pin Placement:

- **create_pin_guide -layer <layer_name> -spacing <value> -pin_name <pin_name>**: Creates a guideline for placing pins, often used before auto-pin placement.
- **place_pins**: Automatically places I/O pins around the die boundary.
    - **-floorplan_side {top | bottom | left | right}**: Specifies which side(s) to place pins on.
    - **-assign_to_boundary_cells true**: Assigns pins to specific boundary cells if they exist.
    - **-spacing <value>**: Sets the spacing between pins.
    - **-justify {left | right | center}**: Justifies pin placement.
    - **-min_io_edge_spacing <value>**: Minimum spacing from the edge of the die.

    - **-fixed_pins {<pin_list>}**: Specifies pins that are already fixed and should not be moved.
    - **-pin_order <order_file>**: Places pins according to a specific order defined in a file.
    - **-snap_to_grid {true | false}**: Aligns pins to the manufacturing grid.
- **create_port -direction in VDD / create_port -direction in VSS**: Define power and ground ports.

5. Power Plan (Initial Power Grid Creation):

While a detailed power plan is often considered a separate stage, the *initial* power grid (VDD/VSS rings around the core and major macros) is often established during floorplanning to guide placement and routing.

- **create_net -power VDD / create_net -ground VSS**: Create the power and ground nets in the design.
- **create_pg_ring_pattern <pattern_name> -horizontal_layers {<layer1> <layer2>} -vertical_layers {<layer1> <layer2>} -width <width> -spacing <spacing>**: Defines patterns for power rings.
- **create_pg_mesh_pattern <pattern_name> -horizontal_layers {<layer1> <layer2>} -vertical_layers {<layer1> <layer2>} -width <width> -spacing <spacing>**: Defines patterns for power stripes (for core area).
- **set_pg_strategy <strategy_name> -rings {<pattern_name>} / -mesh {<pattern_name>}**: Applies the defined patterns as strategies.
- **compile_pg**: Generates the physical power/ground network based on the defined strategies.
- **connect_pg_net -automatic**: Connects power/ground nets to the pins of standard cells and macros.

6. Verification and Reporting (After Floorplan):

- **check_floorplan**: Performs a set of checks to identify issues like:
    - Overlapping macros.
    - Macros outside the core area.
    - I/O pins not on the boundary.
    - Placement blockages overlapping macros.
    - Empty core area.
    - Unreachable areas for standard cells.
- **check_qor**: Provides a quick quality of results check, often used at this stage for area and congestion indicators.
- **report_qor**: More detailed quality of results report.
- **report_floorplan**: Provides a summary of the floorplan dimensions, utilization, and macro/pin details.
- **report_area**: Reports the total die area, core area, and utilization.
- **report_congestion**: Can give an early indication of potential routing congestion hot spots based on macro placement.
- **report_app_options *floorplan***: Reports application options related to floorplanning.

7. Saving the Floorplan:

- **save_mw_cel -as <design_name>_floorplan**: Saves the design database with the completed floorplan to a Milkyway cell. This is the primary way to save the design.
- **write_def <design_name>_floorplan.def**: Writes out the DEF (Design Exchange Format) file. This DEF file contains all the floorplan information (die dimensions, core area, macro placements, I/O pin locations, and initial power grid routing). This DEF is the standard input for the next stage (placement).

Key Goals of Floorplanning:

- **Meet Area Targets:** Achieve the desired chip size and core utilization.
- **Optimize Performance:** Minimize critical path lengths by placing critical blocks close to each other.
- **Manage Congestion:** Distribute blocks to provide sufficient routing channels and avoid hot spots.
- **Ensure Power Integrity:** Establish a robust power distribution network.
- **Enable Testability:** Consider scan chain and test access point requirements.
- **Facilitate Chip Integration:** Plan for interfaces with other blocks or package.

Floorplanning is an iterative process. Early analysis of congestion and estimated timing can lead to adjustments in macro placement, blockages, or even the overall die size.

2. POWER PLANNING

The power plan stage in ICC2 (Synopsys IC Compiler II) is a critical step in the physical design flow, where the power distribution network (PDN) for the chip is created. This involves establishing how power (VDD) and ground (VSS) will be delivered to all standard cells, macros, and other components.

Here's a breakdown of the typical commands used in the power plan stage, from start to end:

1. Initial Setup and Design Loading:

- **read_db <design_name>.db / read_verilog <design_name>.v**: Load the design database (after synthesis) or the Verilog netlist.
- **read_def <floorplan_def_file>.def**: If a floorplan has been created in a previous step, load the DEF file containing the floorplan information (core area, macro placements, I/O pin placements).
- **read_sdc <constraints_file>.sdc**: Load the timing constraints (SDC) file.
- **read_upf <power_intent_file>.upf (Optional, for multi-voltage designs)**: Load the Unified Power Format (UPF) file which defines power domains, retention strategies, and other power-related information.
- **current_design <top_level_design_name>**: Set the current design context.

2. Floorplan Initialization (if not already done):

- **initialize_floorplan -control_type core -core_utilization <value> -shape <shape> -orientation <orientation> -side_length {<x_len> <y_len>} -core_offset {<left_offset> <bottom_offset> <right_offset> <top_offset>}**: If starting floorplan from scratch, this command defines the core area, utilization, shape (rectangular, square, etc.), and placement of the core within the die.
- **create_pin_guide / place_pins**: Commands related to placing I/O pins if they are not already defined by the read_def command.

3. Power Network Creation Commands:

These are the core commands for building the PDN. The typical hierarchy of power distribution is: Power Pads -> Trunks -> Core Rings -> Power Stripes -> Power Rails (Follow Pins) -> Standard Cells.

- **create_port -direction in VDD / create_port -direction in VSS**: Define power and ground ports for the design.
- **create_net -power VDD / create_net -ground VSS**: Create the power and ground nets.
- **derive_pg_connection -power_net VDD -power_pin VDD -ground_net VSS -ground_pin VSS**: Automatically derives power and ground connections based on the defined nets and pins.
- **Creating Power Rings (Core Rings and Trunks):**
    - **create_pg_ring_pattern <pattern_name> -horizontal_layers {<layer1> <layer2>} -vertical_layers {<layer1> <layer2>} -width <width> -spacing <spacing> -offset <offset>**: Defines the pattern for power rings, specifying layers, width, spacing, and offset.

    - **set_pg_strategy <strategy_name> -rings {<pattern_name>}**: Creates a power ring strategy that uses the defined pattern.
    - **compile_pg**: Compiles and creates the power rings according to the defined strategy.
- **Creating Power Stripes:**
    - **create_pg_mesh_pattern <pattern_name> -horizontal_layers {<layer1> <layer2>} -vertical_layers {<layer1> <layer2>} -width <width> -spacing <spacing> -offset <offset>**: Defines the pattern for power stripes (mesh).
    - **set_pg_strategy <strategy_name> -mesh {<pattern_name>}**: Creates a power mesh strategy.
    - **compile_pg**: Compiles and creates the power stripes/mesh.
- **Connecting PG to Standard Cells (Follow Pins):**
    - **connect_pg_net -net VDD [get_pins -physical_context */VDD]**: Connects the VDD net to all VDD pins of standard cells.
    - **connect_pg_net -net VSS [get_pins -physical_context */VSS]**: Connects the VSS net to all VSS pins of standard cells.
    - **connect_pg_net -automatic**: A more automated way to connect PG nets to cell pins.
- **Boundary Cells/Endcaps/Tap Cells:**
    - **create_boundary_cells -left_boundary_cell <cell_name> -right_boundary_cell <cell_name> -prefix ENDCAP**: Places end-cap cells at the boundaries of the design.
    - **create_tap_cells -lib_cell <tap_cell_name> -distance <distance> -pattern stagger -prefix TAPCELL**: Inserts tap cells (for well/substrate connections) at specified distances.
    - **remove_cells [get_cells -filter "ref_name =~ TAP*"] / remove_cells [get_cells -filter "ref_name =~ ENDCAP*"]**: Commands to remove these cells if needed.

4. Power Plan Verification and Analysis:

- **check_pg_drc**: Checks for power-ground related DRC violations.
- **check_pg_missing_vias**: Checks for missing vias in the power network.
- **analyze_fp_rail**: Analyzes the floorplan rails, often used for early IR drop analysis.
- **report_qor**: Can give some early quality of results report.
- **report_app_options *pg***: Reports application options related to power planning.

5. Committing the Power Plan:

- **commit_fp_rail**: Commits the power plan, making it permanent in the design.
- **save_mw_cel -as <design_name>_power_plan**: Saves the design with the completed power plan to a Milkyway cell.
- **write_def <design_name>_power_plan.def**: Writes out the DEF file with the power plan information.

Important Considerations:

- **Technology File and Libraries:** The power plan commands rely heavily on the technology file (.tf) and standard cell libraries (.lib, .db) for layer information, via definitions, and cell characteristics.

- **UPF (Unified Power Format):** For complex multi-voltage designs, the UPF file is crucial for guiding the power network creation, including isolating power domains and setting up voltage shifters.
- **IR Drop Analysis:** While analyze_fp_rail provides early analysis, more detailed IR drop analysis is typically done in later stages (e.g., using PrimePower or RedHawk Analysis Fusion) once the design is more complete.
- **Iterative Process:** Power planning can be an iterative process. You might need to adjust ring/stripe widths, spacing, or placement based on early analysis results to meet power integrity goals.
- **Scripting:** In a real flow, these commands are typically put into Tcl scripts for automation and repeatability.

This list provides a comprehensive overview of common ICC2 commands used in the power plan stage. The exact sequence and specific parameters may vary based on the design complexity, technology node, and specific power requirements.

3. PHYSICAL ONLY CELLS

You're asking about handling "physical only" cells (sometimes called "filler cells" or "deco cells") within ICC2, particularly after the placement stage. These cells don't have logical functionality; their primary purpose is to fill gaps in standard cell rows, provide substrate/well connections (tap cells), or serve as boundary cells (end caps).

Here's a breakdown of ICC2 commands related to placing, managing, and verifying physical-only cells:

1. Context and When Physical Only Cells are Placed:

- **During Placement/Legalization:** The primary placement of these cells usually happens automatically as part of the place_opt or legalize_placement command, especially filler cells.
- **Explicitly by User:** Tap cells and end caps are often inserted explicitly using dedicated commands.
- **Post-Placement Optimization:** Sometimes additional physical-only cells are inserted during post-placement or post-CTS optimization to fix DRVs or improve density.

2. Identifying and Analyzing Physical Only Cells:

- **get_lib_cells -filter "is_filler == true"**: This command retrieves all library cells that are designated as filler cells.
- **get_lib_cells -filter "is_endcap == true"**: Retrieves library cells designated as end-cap cells.
- **get_lib_cells -filter "is_tap == true"**: Retrieves library cells designated as tap cells.
- **get_cells -filter "ref_name == <filler_cell_name>"**: To find instances of a specific filler cell.
- **get_cells -filter "is_physical_only == true"**: This is a more general way to find any cell instance that is designated as "physical only."
- **report_cell <cell_instance_name>**: Reports details about a specific cell instance, including its reference library cell type (which might indicate if it's a filler/tap/endcap).
- **report_qor -cell_count**: Will provide a count of different cell types, including potentially filler/tap/endcap cells, depending on how detailed the report is configured.

3. Inserting Physical Only Cells:

- **For Filler Cells (to fill gaps in rows):**
  - **insert_std_cell_filler -lib_cells {<filler_cell_list>}**: This command inserts filler cells into empty spaces within standard cell rows. It's usually run after placement and legalization, or as part of place_opt's legalization step.
    - place_opt (with legalization enabled, which is default) usually handles this automatically.
    - legalize_placement also performs filler insertion.
  - **set_app_options -name place_opt.legalize.filler_cell_list -value {<filler_cell_list>}**: You can specify the preferred filler cells to be used by the tool during place_opt's legalization.

- **For Tap Cells (for substrate/well contacts):**
    - **create_tap_cells -lib_cell <tap_cell_name> -distance <spacing_value>**: Inserts tap cells at a specified distance along the standard cell rows. This is crucial for latch-up prevention.
        - **-pattern {stagger | line}**: Specifies the pattern of tap cell insertion.
        - **-prefix <prefix_string>**: Adds a prefix to the name of the inserted tap cells.
        - **-create_pg_connection {true | false}**: Connects the power and ground pins of the tap cells to the power network.
        - **-remove_existing_tap_cells**: Removes previously inserted tap cells before inserting new ones.
- **For End Caps / Boundary Cells (at row ends):**
    - **create_boundary_cells -left_boundary_cell <left_end_cap_name> -right_boundary_cell <right_end_cap_name>**: Inserts end-cap cells at the beginning and end of standard cell rows. These often help with DRC closure at row boundaries.
        - **-top_boundary_cell <top_end_cap_name> / -bottom_boundary_cell <bottom_end_cap_name>**: For inserting boundary cells at the top/bottom edges of the core.
        - **-corner_cell <corner_cell_name>**: For inserting cells at row corners.
        - **-prefix <prefix_string>**: Adds a prefix to the name of the inserted cells.
        - **-create_pg_connection {true | false}**: Connects power/ground.

4. Managing (Removing/Modifying) Physical Only Cells:

- **remove_cells [get_cells -filter "ref_name == TAP*"]**: To remove all tap cells (assuming they were named with a "TAP" prefix).
- **remove_cells [get_cells -filter "ref_name == FILL*"]**: To remove all filler cells.
- **remove_cells [get_cells -filter "ref_name == ENDCAP*"]**: To remove all end-cap cells.
- **set_attribute [get_cells <physical_only_cell_instance>] is_dont_touch true**: To prevent the tool from moving or removing a specific physical-only cell.
- **set_attribute [get_cells <physical_only_cell_instance>] is_fixed true**: To fix the location of a specific physical-only cell.

5. Verification Related to Physical Only Cells:

- **check_placement**: This command will report if there are any gaps in standard cell rows that should be filled or if filler cells are placed incorrectly. It also checks for overlaps.
- **check_drc**: Essential for verifying that tap cells, end caps, and filler cells do not create any DRC violations (e.g., spacing issues, well conflicts).
- **check_lvs**: While physical-only cells don't have logical connectivity, they must be correctly recognized and handled by LVS to ensure the layout matches the netlist's representation of them.

Example Flow Snippet for Physical Only Cells:

Tcl
```
# --- Floorplan and Macro Placement done ---
# --- Initial Placement and Legalization done by place_opt ---

# --- Step 1: Create Tap Cells ---
# Assuming you have tap cells like "TAP_1X", "TAP_2X", "TAP_4X" in your library
# Choose the appropriate tap cell based on your technology and design needs
create_tap_cells -lib_cell TAP_1X -distance 200 -pattern stagger -prefix TAP_INST_
# You might want to run this again if you have different tap cell sizes or needs for specific areas

# --- Step 2: Create Boundary Cells (End Caps) ---
# Assuming you have end-cap cells like "ENDCAP_LH", "ENDCAP_RH", "ENDCAP_TOP", "ENDCAP_BOTTOM"
create_boundary_cells \
    -left_boundary_cell ENDCAP_LH \
    -right_boundary_cell ENDCAP_RH \
    -prefix ENDCAP_LR_

# If your library has specific top/bottom/corner endcaps
create_boundary_cells \
    -top_boundary_cell ENDCAP_TOP \
    -bottom_boundary_cell ENDCAP_BOTTOM \
    -corner_cell ENDCAP_CORNER \
    -prefix ENDCAP_TB_

# --- Step 3: Insert Filler Cells to fill remaining gaps ---
# Get a list of your preferred filler cells, usually from smallest to largest
set filler_cells [get_lib_cells {FILL_1 FILL_2 FILL_4 FILL_8}]
insert_std_cell_filler -lib_cells $filler_cells

# --- Step 4: Verification ---
check_placement
check_drc
report_qor # Verify cell counts and no new placement issues

# --- Save the design ---
save_mw_cel -as <design_name>_placed_physical_only_cells
write_def <design_name>_placed_physical_only_cells.def
```

Important Considerations:

- **Library Characterization:** Ensure your technology library accurately characterizes filler, tap, and end-cap cells with is_filler, is_tap, is_endcap attributes and correct power/ground pins.
- **Placement Flow:** place_opt and legalize_placement are highly intelligent and often insert filler cells automatically. You might only need to explicitly run create_tap_cells and create_boundary_cells.

- **DRC Rules:** Tap cells and end caps are critical for meeting manufacturing DRCs, especially related to well and substrate ties.
- **LVS:** Ensure these cells are correctly modeled for LVS verification. They typically have no logical pins but need to be recognized to avoid LVS mismatches.
- **Density:** Filler cells contribute to the overall metal density, which is important for manufacturing.
- **Power Connectivity:** Ensure tap cells and end caps are properly connected to the power and ground nets, as they are essential for distributing these connections throughout the standard cell rows.

4. PLACEMENT AND PLACEMENT OPT

The placement stage in Synopsys ICC2 is where the standard cells (logic gates, flip-flops, etc.) are physically arranged within the core area of the chip, respecting timing, congestion, and power constraints. It's a highly complex and iterative process, often driven by a single comprehensive command (place_opt) that encapsulates many sub-steps.

Here's a breakdown of the ICC2 commands and concepts used in the placement stage, from start to end:

1. Preparation and Setup (Before Placement):

- **read_db <design_name>.db / read_verilog <design_name>.v**: Load the post-synthesis netlist.
- **read_def <floorplan_def_file>.def**: Load the DEF file containing floorplan information (core area, power grid, macro placements, I/O pin placements, and potentially initial blockages).
- **read_sdc <constraints_file>.sdc**: Load the timing constraints (SDC) file. This is crucial as placement is highly timing-driven.
- **read_upf <power_intent_file>.upf (for multi-voltage designs)**: Load the UPF file for power domain information.
- **current_design <top_level_design_name>**: Set the current design context.
- **set_app_options -name place_opt.flow.do_spg -value true (Optional, for SPG flow)**: If you're using a topographical synthesis flow (e.g., with Fusion Compiler), the synthesis tool might output an initial placement DEF. This option tells ICC2 to use that as a starting point.
- **set_app_options -name place_opt.flow.clock.aware_placement -value true**: Enables clock-aware placement, which helps to optimize placement with respect to the clock network.
- **set_app_options -name place_opt.flow.trial_clock_tree -value true**: Enables the tool to use a trial clock tree during placement for better timing estimation and optimization.
- **set_app_options -name place_opt.flow.do_path_opt -value true**: Enables path-based optimization during placement.
- **set_app_options -name place_opt.initial_drc.global_route_based -value true**: Helps the tool to consider global routing information for DRC avoidance during initial placement.

2. Placement Constraints and Pre-Placement (Optional but Recommended):

These commands help guide the placement engine for better QoR.

- **create_placement_blockage -name <blockage_name> -boundary { {llx lly} {urx ury} } -type {hard | soft | partial}**: Defines regions where standard cells should not be placed (hard), or where cell density should be reduced (soft/partial). Useful around macros, sensitive areas, or to address anticipated congestion hot spots.
- **create_keepout_margin -type {hard | soft} -outer {<left> <bottom> <right> <top>} [get_cells <macro_name>]**: Creates a buffer zone around hard macros to prevent standard cells from being placed too close, providing routing channels.

- **create_bound -name <bound_name> -type {soft | hard | exclusive} -boundary { {llx lly} {urx ury} } [get_cells <cells_to_bound>]**: Creates regions where specific cells are encouraged (soft) or forced (hard/exclusive) to be placed. This can be used for critical modules or to group related logic.
- **set_dont_touch [get_cells <cell_names>]**: Prevents the tool from moving or modifying specified cells. Useful for pre-placed cells or cells that require fixed locations.
- **set_app_options -name place_opt.place.congestion_effort -value {high | medium | low}**: Controls the effort the tool puts into resolving congestion during placement. Higher effort leads to better routability but potentially longer runtime.
- **set_app_options -name place_opt.place.density_screen_limit -value <value>**: Sets a density limit for specific regions to control local cell density and reduce congestion.
- **set_max_fanout <value> [current_design]**: Sets the maximum fanout limit for nets, influencing buffer insertion during placement.
- **set_ideal_network [all_fanout -flat -clock_tree]**: Typically applied to clock nets to prevent the tool from inserting buffers or optimizing them during placement, as clock tree synthesis will handle this later.

3. Main Placement Command:

The place_opt command is the workhorse of the placement stage. It encompasses several sub-steps automatically.

- **place_opt**: This is the primary command. By default, it performs a series of steps:
    - **Coarse/Global Placement (initial_place)**: Determines approximate locations for all standard cells based on timing and congestion. Cells might overlap at this stage.
    - **High Fanout Net Synthesis (HFNS) (initial_drc)**: Identifies and synthesizes buffers for high fanout nets (excluding ideal clocks) to improve drivability and transition times. It also addresses logic DRCs.
    - **Preliminary Timing Optimization (initial_opto)**: Performs initial timing-driven optimization, including gate sizing, buffer insertion, and simple netlist changes.
    - **Final Placement (final_place)**: Refines the placement, considering detailed timing and congestion.
    - **Final Timing Optimization (final_opto)**: Performs more comprehensive timing optimization, addressing remaining violations.
    - **Legalization**: Moves overlapping cells to legal, non-overlapping positions on standard cell rows, aligning power and ground pins with the power rails.
- **Common Options for place_opt**:
    - **-effort {high | medium | low}**: Controls the overall effort for placement and optimization.
    - **-congestion_effort {high | medium | low}**: Specific effort for congestion-driven placement.
    - **-timing_effort {high | medium | low}**: Specific effort for timing-driven placement.
    - **-power_effort {high | medium | low}**: Enables power optimization during placement (e.g., leakage power reduction by using lower power cells).
    - **-area_effort {high | medium | low}**: Enables area optimization.

- **-check_preroute**: Checks for prerouted nets and avoids placing cells on top of them.

4. Incremental Placement and Optimization (if needed):

If initial place_opt results in unacceptable QoR (e.g., significant timing violations, high congestion), you might run incremental commands.

- **refine_opt**: This command performs incremental placement and optimization. It's useful for improving timing and congestion without a full re-placement. It also has several sub-stages like initial path optimization, incremental placement, incremental optimization, final placement, and legalization.
- **legalize_placement**: If only legalization is needed after some manual adjustments or minor changes, this command can be run separately.
- **create_placement_blockage -incremental**: Can be used to add blockages to specific congested regions to relieve congestion.
- **remove_placement_blockage <blockage_name>**: Removes previously defined blockages.
- **set_individual_pin_constraints / place_pins**: If I/O pin placement needs refinement or adjustment.

5. Verification and Reporting After Placement:

- **check_placement**: Performs various checks on the placement, such as cell overlaps, cells off-row, unplaced cells, and macro overlaps.
- **check_legality**: Specifically checks for legalization issues.
- **report_qor**: Provides a summary of the Quality of Results, including timing (WNS/TNS), area, cell count, and any design rule violations (DRVs).
- **report_congestion**: Generates a detailed congestion report, showing overflow in global routing cells (GRCs).
- **report_timing**: Generates timing reports to identify setup and hold violations.
- **report_area**: Reports the total cell area and utilization.
- **report_cells**: Reports information about placed cells.
- **report_app_options *place***: Reports application options related to placement.

6. Saving the Design:

- **save_mw_cel -as <design_name>_placement_done**: Saves the design database with the completed placement.
- **write_def <design_name>_placement_done.def**: Writes out the DEF file, which contains the final placement information (cell locations, orientations). This DEF file will be used as input for the next stage (CTS).

Key Concepts in ICC2 Placement:

- **Timing-Driven Placement:** The tool prioritizes placing cells to meet timing constraints, minimizing critical path delays.
- **Congestion-Driven Placement:** The tool attempts to distribute cells evenly to avoid routing congestion hot spots, which can lead to unroutable designs.

- **Multi-objective Optimization:** ICC2 tries to optimize for timing, area, power, and routability concurrently during placement.
- **Legalization:** Ensuring all cells are placed on their respective rows and do not overlap.
- **High Fanout Net Synthesis (HFNS):** Buffering high fanout nets to improve signal integrity and meet fanout limits.
- **Scan Chain Reordering:** Optimizing the physical order of scan flip-flops to minimize scan chain length and improve testability timing.

The placement stage is highly iterative, and designers often need to analyze reports, adjust parameters, and re-run portions of the flow to achieve optimal results.

5. CTS & CTS OPT

Clock Tree Synthesis (CTS) is a crucial stage in physical design, where a balanced and robust clock distribution network is built to ensure all sequential elements (flip-flops, latches) receive the clock signal simultaneously and with minimal skew, transition time, and insertion delay. In Synopsys ICC2, the clock_opt command is the primary workhorse, encapsulating many sub-steps.

Here's a breakdown of the ICC2 commands and concepts used in the CTS stage, from start to end:

1. Preparation and Setup (Inputs to CTS):

- **read_db <design_name>_placed.db / read_def <design_name>_placed.def**: Load the design database or DEF file after the placement stage. All standard cells and macros should be legally placed.
- **read_sdc <constraints_file>.sdc**: Load the timing constraints (SDC) file. This is paramount for CTS as it defines clock definitions, target skew, transition, and latency.
    - **create_clock**: Defines the primary clocks in the design.
    - **create_generated_clock**: Defines clocks derived from primary clocks (e.g., through dividers, multipliers).
    - **set_clock_gating_check**: Specifies how integrated clock gating (ICG) cells should be handled.
    - **set_ideal_network**: Used to mark clock nets as ideal *before* CTS, ensuring the placement tool doesn't optimize them, and CTS will then build the physical network.
    - **set_max_delay -clock_fall -from <clock_source> -to <sink_pin> <max_latency> (Optional)**: Can be used to set target insertion delays for specific clock paths.
    - **set_max_transition**: Critical for defining the maximum allowed transition time on clock nets and sink pins.
    - **set_max_capacitance**: Defines the maximum capacitance allowed on clock nets and sink pins.
    - **set_app_options -name clock_opt.targets.min_period_percent_for_skew -value <value>**: Sets the percentage of clock period to use for skew target (e.g., 5% of clock period).
- **read_upf <power_intent_file>.upf (for multi-voltage designs)**: Ensures proper power domain handling and isolation for clock networks.
- **current_design <top_level_design_name>**: Set the current design context.

2. CTS Configuration and Options (Pre-CTS):

These commands provide guidance and define the desired characteristics of the clock tree.

- **set_clock_tree_options**: This is a versatile command to control various aspects of CTS.
    - **-clock_trees {<clock_name1> <clock_name2>}**: Specifies which clocks to build the tree for. If not specified, it builds for all clocks.
    - **-buffer_list {<buffer_cell1> <buffer_cell2> ...}**: Specifies the list of clock buffer/inverter cells to be used for tree construction.

    - **-routing_rule <rule_name>**: Specifies a non-default routing rule (NDR) for clock nets (e.g., wider wires, specific layers) to reduce resistance and delay.
    - **-layer_list {<layer1> <layer2> ...}**: Specifies preferred routing layers for the clock tree.
    - **-target_skew <value>**: Sets the target skew for the clock tree.
    - **-target_max_transition <value>**: Sets the target maximum transition time for the clock tree.
    - **-target_max_capacitance <value>**: Sets the target maximum capacitance for the clock tree.
    - **-balance_leaf_skew {true | false}**: Controls whether leaf-level skew is balanced.
    - **-min_latency <value> / -max_latency <value>**: Specifies target insertion delay range.
    - **-boundary_cells {<cell_name1> <cell_name2>}**: If you have specific boundary cells (e.g., for power domain crossing) that should not be moved, you can specify them.
- **set_clock_gating_check -output_pins <pin_list> -master_slave_pin <pin_name>**: Further fine-tune ICG cell behavior.
- **set_app_options -name cts.compile.enable_global_route -value true**: Enables the use of global routing information during CTS, which can lead to better routability and timing.
- **set_app_options -name clock_opt.flow.enable_ccd -value true**: Enables Concurrent Clock and Data (CCD) optimization, where the tool optimizes both the clock network and data paths simultaneously for better overall timing closure.
- **set_app_options -name clock_opt.flow.do_post_cts_timing_opt -value true**: Enables post-CTS timing optimization.
- **set_app_options -name clock_opt.cts.area_recovery -value true**: Allows the tool to perform area recovery after CTS, potentially by sizing down buffers that are not critical for timing.
- **set_app_options -name clock_opt.cts.power_recovery -value true**: Allows the tool to perform power recovery after CTS, potentially by replacing high-power cells with lower-power alternatives where timing allows.
- **set_clock_balance_points -consider_for_balancing true -balance_points {<pin1> <pin2>}**: Defines specific points in the clock network that the tool should prioritize for balancing.
- **set_clock_tree_exception**: Defines pins where clock tree construction should be modified.
    - **-stop_pins**: No buffers/inverters inserted beyond these pins.
    - **-ignore_pins**: Excluded from skew/delay balancing, but DRV fixing is still performed.
    - **-exclude_pins**: Only DRV fixing, no balancing or delay optimization.
    - **-float_pins**: Behaves like stop pins, but internal delays are considered.
    - **-through_pins**: DRV fixing and balancing.

3. Main Clock Tree Synthesis Command:

- **clock_opt**: This is the primary command that performs the entire CTS flow. It includes:

    - **Clock Tree Building**: Constructs the physical clock tree using buffers and inverters from the specified list, aiming to meet target skew and insertion delay.
    - **DRV Fixing**: Fixes design rule violations (max transition, max capacitance, max fanout) on clock nets.
    - **Balancing**: Adjusts buffer delays to minimize skew between clock sinks.
    - **Delay Reduction**: Optimizes the clock tree to reduce the overall insertion delay.
    - **Post-CTS Optimization**: Performs timing, area, and power optimization on the clock path and potentially on data paths if CCD is enabled.
    - **Legalization**: Ensures newly inserted clock cells are legally placed.
- **Common Options for clock_opt**:
    - **-effort {high | medium | low}**: Controls the overall effort for CTS and optimization.
    - **-stage_only**: Runs only the CTS stage, without post-CTS optimization. Useful for debugging or incremental flows.
    - **-no_common_point_optimization**: Prevents common point optimization, which can be useful for specific clock architectures.

4. Incremental CTS and Optimization (if needed):

If initial clock_opt results in unacceptable QoR, incremental commands can be used.

- **refine_clock_tree**: Performs incremental clock tree optimization.
- **refine_opt**: Can be used for broader incremental timing and power optimization after CTS, often with -hold effort to address hold violations.
- **set_app_options -name clock_opt.hold.effort -value high**: Increases effort for hold timing fixing during clock_opt or refine_opt.
- **route_clock_nets**: If you only need to route specific clock nets after clock_opt or for incremental routing.

5. Verification and Reporting After CTS:

- **check_clock_tree**: Verifies the integrity of the clock tree, checking for unbuffered clocks, loops, and other issues.
- **report_clock_tree**: Provides a detailed report on the clock tree, including:
    - Clock source, sinks, and hierarchy.
    - Insertion delay, skew, and transition times for each clock.
    - Buffer/inverter counts.
    - Path details from source to sinks.
- **report_timing -path_type full_clock_expanded**: Provides detailed timing reports, specifically expanding the clock path to see the full clock network delay.
- **report_qor**: Provides an overall Quality of Results report, including setup and hold timing, area, and power. Pay close attention to hold timing after CTS, as it's a common post-CTS issue.
- **report_power**: Reports the power consumption of the design, including clock power.
- **report_congestion**: Checks for any new congestion introduced by the clock tree routing.
- **check_drc**: Performs design rule checks (spacing, width, via issues) on the newly routed clock nets.

- **report_app_options *clock_opt***: Reports application options related to clock tree synthesis.

6. Saving the Design:

- **save_mw_cel -as <design_name>_cts_done**: Saves the design database with the completed clock tree.
- **write_def <design_name>_cts_done.def**: Writes out the DEF file, containing the updated placement of clock cells and the physical routing of clock nets. This DEF is the input for the routing stage.

Key Goals of CTS:

- **Minimize Skew:** Ensure clock signals arrive at all sequential elements as close to simultaneously as possible.
- **Minimize Insertion Delay:** Reduce the time it takes for the clock signal to travel from the source to the sinks.
- **Meet Transition Time Constraints:** Ensure the rise and fall times of clock signals are within specified limits.
- **Meet Capacitance Constraints:** Ensure the load on clock buffers does not exceed their drive capability.
- **Maintain Duty Cycle:** Preserve the clock signal's high/low time ratio.
- **Optimize Power:** Minimize the power consumed by the clock network, as it can be a significant portion of total chip power.
- **Resolve Hold Violations:** CTS often introduces hold violations due to delay changes in the clock path; these are typically fixed in post-CTS optimization or a dedicated hold fixing stage.

CTS is a highly automated and optimized stage, but proper setup, constraint definition, and post-CTS analysis are crucial for achieving a robust and high-performance design.

6. ROUTING & ROUTING OPT

The routing stage in Synopsys ICC2 is where the physical interconnections (wires) between all the placed standard cells, macros, and I/O pins are created. It's a highly complex process that must adhere to a vast set of design rules (DRC), meet timing constraints, and minimize wire length for better power and performance.

Similar to placement and CTS, ICC2 provides a top-level command, route_opt, that orchestrates the entire routing flow.

Here's a breakdown of the ICC2 commands and concepts used in the routing stage, from start to end:

1. Preparation and Setup (Inputs to Routing):

- **read_db <design_name>_cts_done.db / read_def <design_name>_cts_done.def**: Load the design database or DEF file after the Clock Tree Synthesis (CTS) stage. This includes the placed cells, routed clock networks, and power/ground distribution.
- **read_sdc <constraints_file>.sdc**: Load the timing constraints (SDC) file. Timing closure is a primary goal during routing.
- **read_upf <power_intent_file>.upf (for multi-voltage designs)**: Essential for proper power domain handling during routing, especially for level shifters and isolation cells.
- **current_design <top_level_design_name>**: Set the current design context.
- **set_app_options -name route_opt.flow.do_global_route_only -value true (Optional, for initial global route only)**: If you want to perform only global routing before detailed routing, set this.
- **set_app_options -name route_opt.common.effort -value {high | medium | low}**: Sets the overall effort for routing and optimization. High effort generally leads to better QoR but longer runtime.
- **set_app_options -name route_opt.flow.optimize_ndr -value true**: Enables optimization for non-default routing rules.
- **set_app_options -name route_opt.drc_efffort -value {high | medium | low}**: Controls the effort to resolve DRC violations during routing.

2. Routing Constraints and Pre-Routing (Optional but Recommended):

These commands help guide the routing engine for better results and manufacturability.

- **define_routing_rule <rule_name> -widths {<layer> <width> ...} -spacings {<layer> <spacing> ...}**: Defines non-default routing rules (NDRs) for specific nets or groups of nets. NDRs specify wider wires or larger spacings to reduce resistance, capacitance, or crosstalk, typically used for critical signals or power nets.
- **set_routing_rule -rule <rule_name> [get_nets <net_list>]**: Applies a defined NDR to a specific set of nets.
- **set_routing_rule -min_routing_layer <layer_name> -max_routing_layer <layer_name> [get_nets <net_list>]**: Constrains specific nets to route only on certain layers. Useful for critical paths, shielding, or avoiding noisy layers.
- **create_routing_blockage -boundary { {llx lly} {urx ury} } -layers {<layer_list>} -type {hard | soft | partial}**: Creates regions where routing is restricted or forbidden

on specific layers. Useful around sensitive IP, analog blocks, or to prevent routing over certain features.

- **create_routing_guide -boundary { {llx lly} {urx ury} } -layers {<layer_list>} -type {vertical | horizontal}**: Creates preferred routing directions within a region for specific layers.
- **set_crosstalk_options -max_crosstalk_delay <value> -max_crosstalk_noise <value>**: Sets targets for crosstalk-driven optimization.
- **set_min_routing_layer <layer_name> / set_max_routing_layer <layer_name>**: Sets default minimum and maximum routing layers for the entire design.
- **set_routing_layer_direction <layer_name> {horizontal | vertical}**: Specifies the preferred routing direction for each metal layer in the technology file (usually defined in the tech file itself).

3. Main Routing Command:

The route_opt command is the main command that drives the routing process. It's an umbrella command that orchestrates global routing, track assignment, and detailed routing, along with various optimizations.

- **route_opt**: This command performs the complete routing flow. It encompasses:
    - **Global Routing (route_global)**: Determines the approximate paths for all nets across the design, breaking the chip into global routing cells (GRCs) and assigning nets to GRCs. This stage primarily focuses on routability and congestion.
    - **Track Assignment (route_track_assign)**: Assigns nets to specific routing tracks within the GRCs.
    - **Detailed Routing (route_detail)**: Performs the final, precise routing of all nets, adhering to all DRC rules (minimum width, spacing, via rules, etc.) and optimizing for timing, signal integrity, and manufacturing. This is where the actual polygons for wires and vias are created.
    - **Post-Route Optimization (post_route_opt)**: Performs various optimization steps after detailed routing, including:
        - **DRC Fixing**: Addresses any remaining design rule violations.
        - **Timing Optimization**: Further sizes cells, inserts buffers/inverters, and adjusts wires to meet timing constraints (setup and hold). This often involves eco-routing.
        - **Crosstalk Reduction**: Inserts shielding, changes spacing, or re-routes nets to mitigate crosstalk.
        - **Antenna Fixing**: Inserts antenna diodes to prevent damage during manufacturing.
        - **Power/Ground Via Optimization**: Ensures robust PG connections.
        - **Metal Fill**: Adds dummy metal shapes to meet density requirements for manufacturing (often a separate step or part of final signoff).
- **Common Options for route_opt**:
    - **-congestion_effort {high | medium | low}**: Controls the effort to resolve congestion during routing.
    - **-timing_effort {high | medium | low}**: Controls the effort for timing-driven routing and optimization.
    - **-drc_effort {high | medium | low}**: Controls the effort for resolving DRC violations.

    - **-power_effort {high | medium | low}**: Enables power optimization during routing (e.g., minimizing wire length for less dynamic power).
    - **-area_effort {high | medium | low}**: Enables area optimization during routing.
    - **-enable_multi_cpu**: Utilizes multiple CPU cores for faster execution.
    - **-concurrent_logic_and_route_opt**: Enables concurrent optimization of logic and routing for better convergence.
    - **-auto_wire_width**: Allows the tool to vary wire widths to optimize for timing or other objectives.
    - **-fix_hold**: Specifically focuses on fixing hold violations.

4. Incremental Routing and Optimization (if needed):

After the initial route_opt, there might be remaining timing violations, DRVs, or minor issues that require incremental fixes.

- **route_global**: Can be run incrementally to improve global routing if congestion is severe.
- **route_detail**: Can be run incrementally (route_detail -incremental true) to fix specific DRCs, shorts, or open nets.
- **route_eco**: Used for engineering change order (ECO) routing, where only a subset of nets or specific changes are routed to fix problems.
    - **-open_net_driven true**: Routes only unrouted nets.
    - **-short_driven true**: Routes only nets involved in shorts.
    - **-timing_driven**: Routes nets to fix timing violations.
    - **-drc_driven**: Routes nets to fix DRC violations.
- **refine_route**: Performs incremental routing and timing optimization.
- **refine_opt -post_route**: A general post-route optimization command that can perform cell sizing, buffering, and wire optimization to clean up timing and DRVs.
- **remove_routes -nets <net_list>**: Removes routes for specific nets, allowing them to be re-routed.
- **remove_via_cut_blockage / create_via_cut_blockage**: Used to manage via-related blockages.

5. Verification and Reporting After Routing:

This is a critical phase to ensure the design is clean and meets all requirements before signoff.

- **check_routes**: Performs comprehensive checks on the routing, identifying:
    - **Shorts**: Wires connected incorrectly.
    - **Opens**: Unconnected pins or nets.
    - **DRC Violations**: Violations of width, spacing, via rules, etc.
    - **Antenna Violations**: Issues related to large metal areas collecting charge during manufacturing.
- **check_drc**: A general DRC checker, often used as a final sanity check.
- **report_qor**: Provides the final Quality of Results summary, including timing (WNS/TNS, setup/hold), area, power, and DRVs. This is a key report for signoff.
- **report_timing -scenario <scenario_name> -path_type full -nworst <N>**: Generates detailed timing reports for critical paths.

- **report_net -physical**: Provides detailed information on net routing, including length, layers used, and via count.
- **report_drc -rule_names {<rule_name_list>}**: Reports specific DRC violations.
- **report_crosstalk**: Reports crosstalk-related delays and noise.
- **report_power**: Provides detailed power consumption analysis.
- **report_congestion**: Checks for any remaining routing congestion.
- **report_app_options *route***: Reports application options related to routing.

6. Saving the Design:

- **save_mw_cel -as <design_name>_routed**: Saves the design database with the fully routed design.
- **write_def <design_name>_routed.def**: Writes out the DEF file containing the complete physical layout information, including cell placements, macro placements, and all signal, clock, and power/ground routing. This DEF is crucial for GDSII stream out and final signoff checks (DRC/LVS).
- **write_gds (or a similar command in the signoff tool)**: While not directly an ICC2 routing command, the write_def output from routing is ultimately used to generate the GDSII file, which is the final layout representation for manufacturing.

Key Goals of Routing:

- **100% Routability:** All nets must be connected without any open circuits or shorts.
- **DRC Clean:** The layout must adhere to all foundry design rules to ensure manufacturability.
- **Timing Closure:** All setup and hold timing constraints must be met.
- **Minimize Crosstalk:** Reduce signal interference between adjacent wires.
- **Minimize Wire Length:** Reduces resistance and capacitance, leading to lower delay and power consumption.
- **Antenna Rule Compliance:** Prevent damage to gates during the manufacturing process.
- **Power Integrity:** Ensure the power distribution network is robust and meets IR drop requirements.

The routing stage is often the most challenging and time-consuming part of the physical design flow, requiring significant iteration and attention to detail to achieve a manufacturing-ready design.

7. SIGN-OFF

The sign-off stage in VLSI physical design is the final and most critical phase before tape-out to manufacturing. Its purpose is to ensure that the design meets all functional, performance, power, area, and manufacturability requirements across all specified operating conditions (PVT corners). While ICC2 is primarily an implementation tool (placement, CTS, routing), it integrates with or generates outputs for various sign-off analysis tools.

Therefore, the "sign-off commands in ICC2" refer to the commands used *within ICC2* to prepare the design for these sign-off checks, or to perform preliminary checks that are indicative of sign-off readiness. The actual sign-off checks are usually performed by dedicated, industry-standard sign-off tools (e.g., Synopsys PrimeTime for STA, Synopsys IC Validator or Mentor Calibre for PV, Synopsys PrimeRail for power integrity).

Here's a breakdown of the ICC2 commands and the associated sign-off checks they relate to:

1. Finalizing the Design in ICC2:

- **save_mw_cel -as <design_name>_final_routed**: Save the final routed design database. This is the golden database from which all sign-off collateral will be generated.
- **write_def <design_name>_final.def**: Write out the final DEF (Design Exchange Format) file. This DEF contains the complete physical layout information (cell placement, macro placement, and all signal, clock, and power/ground routing). It's a key input for physical verification and extraction tools.
- **write_verilog -physical_only -output <design_name>_physical_only.v**: Writes out a "physical-only" Verilog netlist, containing only the instances and connectivity related to the physical layout, without functional logic. This is used for LVS.
- **write_parasitic_data -output <design_name>.spef / write_qrc_data -output <design_name>.qrc_data**: Generates parasitic extraction files (SPEF or QRC database). These files capture the resistance and capacitance of all interconnects (wires and vias) from the physical layout. This is crucial for accurate static timing analysis and power integrity analysis.
    - **set_extraction_options**: Control the accuracy and scope of parasitic extraction (e.g., considering coupling capacitance, specific layers).
    - **extract_parasitics**: Performs the actual parasitic extraction within ICC2 (though often a more dedicated tool like StarRC is used for final sign-off extraction).

2. Static Timing Analysis (STA) Preparation and Reporting:

While PrimeTime is the sign-off STA tool, ICC2 performs integrated timing analysis and optimization throughout the flow. Final timing checks in ICC2 are a strong indicator of sign-off timing readiness.

- **read_sdc <constraints_file>.sdc**: Load the SDC file, ensuring all timing constraints (clocks, exceptions, I/O delays, operating conditions) are correctly applied for all sign-off corners.
- **read_parasitic_data <design_name>.spef**: Load the extracted parasitics (if done external to ICC2 or from a previous run).

- **set_scenario_status -active true -timing true -power true -signoff true [get_scenarios *]**: Ensure all timing scenarios (multi-mode, multi-corner) are enabled for analysis.
- **update_timing**: Updates the timing engine with the latest design and parasitic information.
- **report_qor**: The most common high-level command to get a summary of timing (WNS, TNS for setup and hold), area, and power. This is checked extensively.
- **report_timing -path_type full -nworst <N> -delay_type {max | min} -input_pins -output_pins -exceptions -fields {all}**: Generates detailed timing reports for critical setup (max delay) and hold (min delay) paths across all corners.
    - **report_timing -check_type {setup | hold | recovery | removal | pulse_width | nochange}**: Reports specific timing checks.
    - **report_timing -group <path_group_name>**: Reports timing for specific path groups.
- **check_timing**: Performs a set of checks to ensure timing analysis can be performed correctly (e.g., unconstrained paths, missing clocks).
- **report_clock_tree**: Verifies clock network characteristics like skew, insertion delay, and transition.
- **report_constraint -all_violators**: Reports all timing constraints that are violated.
- **write_sdf -output <design_name>.sdf**: Generates an SDF (Standard Delay Format) file, which contains delay information that can be used by other tools for gate-level simulation or post-layout verification.

3. Design Rule Checking (DRC):

While ICC2 has integrated DRC capabilities (check_drc), the final sign-off DRC is typically performed by a dedicated physical verification (PV) tool (e.g., Synopsys IC Validator, Mentor Calibre) using the foundry's golden rule deck. ICC2's check_drc is for on-the-fly verification during implementation.

- **check_drc**: Runs internal DRC checks based on the loaded technology file. This is usually run extensively after routing.
- **report_drc -rule_names {<list_of_rules>}**: Reports specific DRC violations.
- **fix_drc**: Attempts to automatically fix certain DRC violations (usually run during routing optimization, not strictly sign-off).
- **write_drc_error_file <design_name>.drc_errors**: Writes out DRC error reports for analysis.

4. Layout Versus Schematic (LVS):

LVS ensures that the physical layout (GDSII) matches the logical netlist (Verilog). Like DRC, this is primarily done with dedicated PV tools (IC Validator, Calibre). ICC2 can perform a basic LVS check (check_lvs).

- **check_lvs -hdl <reference_netlist.v> -top <top_module_name> -connectivity_checks**: Performs a connectivity check between the current physical design in ICC2 and a reference Verilog netlist. This is a preliminary check.
- **check_lvs -max_error 0 -checks {short open}**: Can be used to check for specific LVS issues like shorts or opens.
- **report_lvs**: Reports LVS mismatches.

5. Power Integrity (IR Drop/EM) Analysis:

While ICC2 can perform basic power grid checks, detailed static and dynamic IR drop and electromigration (EM) analysis are performed by specialized tools (e.g., Synopsys PrimeRail, Ansys RedHawk).

- **check_pg_drc**: Checks for power/ground network connectivity issues and basic DRCs.
- **check_pg_missing_vias**: Checks for missing vias in the power network.
- **report_pg_path**: Can trace power paths.
- **report_power**: Provides power consumption reports (leakage and dynamic). This report in ICC2 is based on cell characterization data and estimated activity, providing an estimate. Sign-off power analysis uses detailed activity files (SAIF/VCD/FSDB) and dedicated tools.

6. Reliability Analysis (Antenna, ESD):

- **check_antenna**: Checks for antenna violations on signal nets. These occur during manufacturing when metal layers act as "antennas" and accumulate charge, potentially damaging gate oxides.
- **fix_antenna**: Attempts to insert antenna diodes to mitigate violations.
- **report_antenna**: Reports antenna violations.
- **ESD (Electrostatic Discharge) checks:** While not direct ICC2 commands for analysis, ensuring ESD cells are properly placed and connected is part of the layout. Verification is done by PV tools.

7. Other Sign-off Related Data Generation:

- **write_lib_cells -type {lef} -output <design_name>.lef**: Writes out a LEF (Library Exchange Format) of the current design. This might be used for higher-level integration or verification.
- **write_oa_data -output <design_name>.oa**: Writes out the design in OpenAccess format, which is a common database format for various EDA tools.
- **write_gds_stream -output <design_name>.gds -type gds**: **This is the ultimate sign-off output.** This command is typically run from a dedicated physical verification tool (like IC Validator or Calibre) that consumes the DEF, GDS libraries, and technology files to generate the final GDSII stream for fabrication. While ICC2 might have a write_gds equivalent, it's usually the PV tool's responsibility for the final, certified GDSII.

Sign-off Flow Summary (typically outside of ICC2, but driven by ICC2 outputs):

1. **Extract Parasitics:** Run StarRC (or similar) on write_def output to generate SPEF.
2. **Static Timing Analysis (STA):** Run PrimeTime with write_sdc, extracted SPEF, and libraries for all PVT corners. Generate final setup/hold/DRV/AOCV reports.
3. **Physical Verification (PV):**
    - **DRC (Design Rule Checking):** Run IC Validator/Calibre on write_def (or write_gds) and foundry rule deck.
    - **LVS (Layout Versus Schematic):** Run IC Validator/Calibre on write_def (or write_gds) and reference netlist.

    - **Antenna:** Run IC Validator/Calibre.
    - **Metal Fill:** Add dummy metal shapes to meet density requirements.
4. **Power Integrity (PI) Analysis:** Run PrimeRail/RedHawk with write_def, extracted SPEF, activity files (VCD/FSDB), and libraries for IR drop and EM analysis.
5. **Formal Verification (Equivalence Checking):** Run Formality/Conformal to formally verify that the final physical netlist (write_verilog) is logically equivalent to the golden RTL. This ensures no unintended logical changes occurred during physical implementation.
6. **Reliability (ESD, EOS, etc.):** Further checks performed by specialized tools.
7. **DFM (Design for Manufacturability):** Checks to improve yield and reduce variability.

In essence, while ICC2's commands focus on the implementation phase, its sign-off-related commands are crucial for generating the necessary data (.def, .spef, .v) that feeds into the dedicated sign-off tools, which then perform the final, comprehensive verification before the design is deemed ready for manufacturing.